



南開大學
Nankai University

南 開 大 學

計 算 機 學 院

計算機系統前沿實驗報告

LevelDB 性能測試與機制探究

王 燁

年級：2024 級

專業：計算機科學與技術

指導教師：崔立驍

2026 年 5 月 3 日

目录

一、实验目的	1
二、测试环境说明	1
(一) Task 1	1
(二) Task 2 & Task 3	1
三、测试流程	1
(一) 前期准备	1
1. LevelDB 本地迁移	2
2. WSL2 参数配置	2
3. VS Code 编译环境调试	2
(二) 任务一: LevelDB 核心参数调优与性能对比	3
(三) 任务二: 长尾延迟测量与 Compaction 现象观察	5
(四) 任务三: 应用层 KV Cache 与底层 Block Cache 的设计与对比	5
四、测试分析及结论	7
(一) 任务一	7
1. write_buffer_size 参数对比	7
2. block_size 参数对比	8
3. bloom_filter 参数对比	9
(二) 任务二	9
(三) 任务三	10
1. Block Cache 的基本原理	11
2. 点查 (Point Get) 场景分析	12
3. 范围查找 (Range Scan) 场景分析	12
4. 读写混合负载 (Read-Write Mix) 场景分析	13
五、总结	13

一、 实验目的

1. 深入理解 LSM-Tree (Log-Structured Merge-Tree) 的架构特点及其在 LevelDB 中的具体实现。
2. 掌握数据库核心参数对系统吞吐量和延迟的影响。
3. 通过实测捕捉并理解 Compaction 机制带来的“写毛刺”(Write Stalls) 现象。
4. 探究不同缓存体系(应用层 KV Cache、leveldb 底层 Block Cache) 的交互与性能差异。

二、 测试环境说明

(一) Task 1

- 操作系统: Ubuntu 22.04 LTS (基于 aliyun 服务器)
- CPU: 2 核 (vCPU)
- 内存: 8GB
- 编程语言: C++ 11
- 核心依赖: LevelDB 代码库 (<https://github.com/google/leveldb>)、pthread 库
- 测试工具: 基于 leveldb_test.cpp 修改的 C++ 压测框架 + LevelDB_bench

(二) Task 2 & Task 3

1. 操作系统: Ubuntu 22.04 LTS (基于 WSL2)
2. CPU: 13th Gen Intel(R) Core(TM) i7-13650HX (WSL2 分配 12 核)
3. 内存: 11GB (WSL2 分配)
4. 编程语言: C++ 11
5. 核心依赖: LevelDB 代码库 (<https://github.com/google/leveldb>)、pthread 库
6. 测试工具: 基于 leveldb_test.cpp 修改的 C++ 压测框架

三、 测试流程

(一) 前期准备

完成测试所需要进行的前期准备包括: LevelDB 本地迁移, WSL2 参数配置, VS Code 编译环境调试。

1. LevelDB 本地迁移

受限于网络代理，这里选择了本地迁移的方法。利用 fastgithub.exe 加速 github 后，打开 Windows PowerShell，执行指令：

```
git clone --recurse-submodules https://github.com/google/leveldb.git
```

将 LevelDB 仓库克隆至本地，随后复制进入 WSL2 下实验目录，然后在终端执行：

```
cd ~/leveldb
mkdir -p build && cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j$(nproc)
sudo make install
sudo ldconfig
```

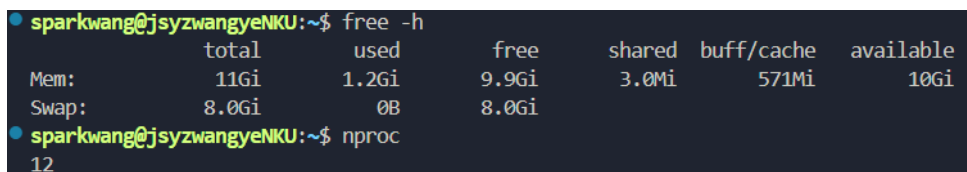
这样我们便顺利完成了 LevelDB 的相关配置，为后续实验打下基础。

2. WSL2 参数配置

注意到在任务二中，要求**数据集大小必须大于系统可用物理内存的 2 倍，以确保数据真实落盘并触发频繁的 Compaction**，同时为了提高 WSL2 对原生硬件的利用率，在用户目录下编写 .wslconfig 文件，配置 WSL2 相关参数：

```
[wsl2]
memory=12GB
processors=12
swap=8GB
pageReporting=false
guiApplications=false
```

随后打开 Windows PowerShell，执行 wsl --shutdown，待 30s 后重启，在 WSL2 终端执行 free -h 和 nproc 指令，结果显示：



```
sparkwang@jsyzwangyeNKU:~$ free -h
               total        used        free       shared    buff/cache   available
Mem:            11Gi         1.2Gi         9.9Gi         3.0Mi        571Mi        10Gi
Swap:           8.0Gi           0B         8.0Gi
sparkwang@jsyzwangyeNKU:~$ nproc
12
```

图 1: WSL2 successfully configured

这说明我们成功完成了 WSL2 参数的配置。

3. VS Code 编译环境调试

我们在 VS Code 中与 WSL2-Ubuntu 22.04 构建远程连接，随后创建用于三个测试任务的 cpp 文件，分别为 task1_benchmark.cpp, task2_latency.cpp 和 task3_cache.cpp，并编写 CMakeLists.txt 完成统一编译。限于篇幅，源代码不在报告中展示，而是与报告整体打包，可以完整运行。

编写完成后，在 WSL2 终端执行指令：

```
cd ~/leveldb_lab
mkdir -p build && cd build
cmake ..
make -j$(nproc)
```

这样我们便完成了编译环境调试，至此所有关于 LevelDB 性能测试的前期准备全部完成。

(二) 任务一：LevelDB 核心参数调优与性能对比

LevelDB 提供了丰富的 `leveldb::Options` 供开发者调优。不同的业务负载（读密集型 vs 写密集型）需要不同的参数组合。在任务一中，我们需要统一控制单次 Value 大小为 1KB，总数据量不低于 2GB，通过分别调整 `write_buffer_size` (4MB、16MB、64MB)、`block_size` (4KB、16KB、64KB)、`filter_policy` (on and off) 三组核心参数，记录并对比不同配置下系统的 QPS 与延迟表现。

我们基于提供的 `leveldb_test.cpp` 文件进行改编，重构为 `task1_benchmark.cpp`，结合 LevelDB 官方测试工具 `db_bench`，分开执行三组测试，并将结果重定向至 `task1_result.txt` 文件用于数据分析。

编译成功后，在云服务器终端执行指令：

```
sudo rm -rf /tmp/leveldb_task1
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
sleep 3
./task1_benchmark --test=rand_write --write_buffer_size=4 >> task1_result.txt 2>&1

sudo rm -rf /tmp/leveldb_task1
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
sleep 3
./task1_benchmark --test=rand_write --write_buffer_size=16 >> task1_result.txt 2>&1

sudo rm -rf /tmp/leveldb_task1
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
sleep 3
./task1_benchmark --test=rand_write --write_buffer_size=64 >> task1_result.txt 2>&1

sudo rm -rf /tmp/leveldb_task1
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
sleep 3
./task1_benchmark --test=rand_read --block_size=4 >> task1_result.txt 2>&1

sudo rm -rf /tmp/leveldb_task1
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
sleep 3
./task1_benchmark --test=rand_read --block_size=16 >> task1_result.txt 2>&1

sudo rm -rf /tmp/leveldb_task1
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
```

```

sleep 3
./task1_benchmark --test=rand_read --block_size=64 >> task1_result.txt 2>&1

sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
sleep 3

./db_bench --benchmarks=fillrandom,readrandom,readmissing \
--num=5000000 \
--reads=5000000 \
--value_size=1000 \
--bloom_bits=-1 \
--threads=2 \
--db=/tmp/leveldb_bench_off

sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
sleep 3

./db_bench --benchmarks=fillrandom,readrandom,readmissing \
--num=5000000 \
--reads=5000000 \
--value_size=1000 \
--bloom_bits=10 \
--threads=2 \
--db=/tmp/leveldb_bench_on

```

测试结果如下:

表 1: write_buffer_size 对随机写性能的影响

MemTable 大小	QPS	平均延迟 (ms)	性能变化
4MB	43632.9	0.0229	
16MB	88164.9	0.011	+102.1%
64MB	137573	0.007	+215.3%

表 2: block_size 对随机读性能的影响

SSTable 大小	QPS	平均延迟 (ms)	性能变化
4KB	44942.5	0.022	
16KB	36409.7	0.027	-18.9%
64KB	36490.2	0.027	-18.8%

表 3: bloom_filter 对随机读性能的影响

测试阶段	布隆过滤器	平均延迟 ($\mu\text{s}/\text{op}$)	QPS
fillrandom	关闭	129.777	7,705
	开启 (10 bits)	130.334	7,673
readrandom	关闭	27.278	36,660
	开启 (10 bits)	28.218	35,438
readmissing	关闭	22.911	43,647
	开启 (10 bits)	12.552	79,668

(三) 任务二：长尾延迟测量与 Compaction 现象观察

LSM-Tree 在后台进行 Compaction 时，会消耗大量的磁盘 I/O 和 CPU 资源，甚至阻塞前端写入，导致严重的尾延迟（Tail Latency）上升。在任务二中，我们需要编写一段 C++ 压测程序，向 LevelDB 持续进行高速随机写入，要求数据集大小必须大于系统可用物理内存的 2 倍，统计并输出写入延迟的 P50、P90、P99 和 P99.9 分位值。

我们通过自编的 task2_latncy.cpp，由于 WSL2 分配了 11GB 的内存，设定数据集大小为 2500 万条 (25GB)，执行测试，并将结果重定向至 task2_result.txt 文件用于数据分析。

表 4: 随机写延迟分位值

延迟分位	延迟
P50	1.419 μs
P90	1066.49 μs
P99	1170.99 μs
P99.9	3132.55 μs

(四) 任务三：应用层 KV Cache 与底层 Block Cache 的设计与对比

LevelDB 内部提供了基于 LRU 的 Block Cache（缓存解压后的数据块）。在实际工程中，我们常常还会在 leveldb 之上再封装一层应用级的 KV Cache（键值对为粒度的缓存，如使用 C++ std::unordered_map 加读写锁实现）。在任务三中，我们首先要实现一个具有容量上限的简单 LRU KV Cache，作为 LevelDB 的前置代理。随后，使用符合 Zipf 分布（模拟二八定律热点数据）的查询请求，分别对比高频随机读，范围查询以及读写混合负载（设定 80% 读 + 20% 写）下的 QPS 表现。

首先，基于测试要求，编写 lru_cache.h 文件，作为 LevelDB 的前置代理：

```
#ifndef LRU_CACHE_H
#define LRU_CACHE_H

#include <unordered_map>
#include <list>
#include <string>
#include <mutex>
```

```
class LRUCache {
private:
    size_t capacity_;
    std::list<std::pair<std::string, std::string>> lru_list_;
    std::unordered_map<std::string, decltype(lru_list_.begin())> cache_map_;
    mutable std::mutex mutex_;

public:
    explicit LRUCache(size_t capacity) : capacity_(capacity) {}

    bool Get(const std::string& key, std::string* value) {
        std::lock_guard<std::mutex> lock(mutex_);
        auto it = cache_map_.find(key);
        if (it == cache_map_.end()) {
            return false;
        }
        lru_list_.splice(lru_list_.begin(), lru_list_, it->second);
        *value = it->second->second;
        return true;
    }

    void Put(const std::string& key, const std::string& value) {
        std::lock_guard<std::mutex> lock(mutex_);
        auto it = cache_map_.find(key);
        if (it != cache_map_.end()) {
            it->second->second = value;
            lru_list_.splice(lru_list_.begin(), lru_list_, it->second);
            return;
        }
        if (cache_map_.size() >= capacity_) {
            auto last = lru_list_.end();
            last--;
            cache_map_.erase(last->first);
            lru_list_.pop_back();
        }
        lru_list_.emplace_front(key, value);
        cache_map_[key] = lru_list_.begin();
    }

    size_t Size() const {
        std::lock_guard<std::mutex> lock(mutex_);
        return cache_map_.size();
    }
};

#endif
```

通过构建哈希表配合双向链表的数据结构，当缓存满时，直接从链表末尾移除“最久未使用”

的元素，并在哈希表中同步删除，所有操作都是 $O(1)$ 时间复杂度，实现 LRU 淘汰策略。Get 时先在哈希表中查找，命中的话直接将对应节点移动到链表头部（表示最近被访问）并返回值，未命中则返回 false；Put 时同样先查哈希表，若键已存在则更新值并将节点移至头部，若不存在则检查容量，当缓存已满时先淘汰链表尾部的“最久未使用”节点（从哈希表中删除并弹出链表），随后将新键值对插入链表头部并建立哈希索引，全程通过互斥锁保护以保证线程安全。

随后，执行负载测试，测试结果如下：

表 5: Block Cache 与 KV Cache 性能对比

测试负载	缓存类型	总操作数	耗时 (s)	QPS	KV Cache 命中率
Point Get	Block Cache	2,000,000	3.779	529,241	—
	KV Cache	2,000,000	3.064	652,795	93.45%
Range Scan	Block Cache	100	0.316	316.3	—
	KV Cache	100	0.307	325.4	0.00%
Read-Write Mix	Block Cache	2,000,000	4.902	408,029	—
	KV Cache	2,000,000	5.511	362,902	93.42%

四、测试分析及结论

（一）任务一

1. write_buffer_size 参数对比

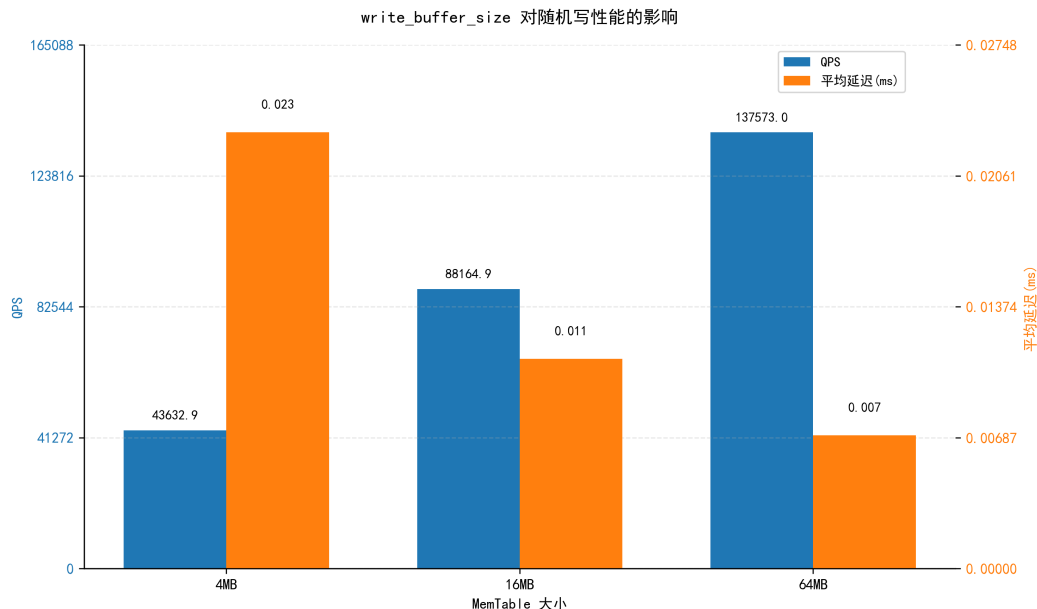


图 2: Task1 write_buffer_size

根据对比图可以发现，当 write_buffer_size 逐渐增大时，QPS 近似呈现线性增长的趋势，平均延迟呈现比例下降的趋势，这是符合 LevelDB 基本架构的。

在 LevelDB 中，最新写入的数据都会保存到 memtable 中。每个 LevelDB 实例最多维护两个 memtable，分别为可读可写的 memtable 和只读的 Immutable MemTable。

当 memtable 的大小超过 write_buffer_size 时，LevelDB 就会将其切换成只读的 Immutable MemTable 并生成新的 memtable，同时触发一个 minor compaction 将其持久化为 Level 0 层的 SST 文件。

write_buffer_size 决定了一次性写入磁盘的数据量。当我们扩大 write_buffer_size 时，在内存中可以积累更多的数据后才触发一次 Compaction，由此大大减少了刷盘次数，降低了 Compaction 的频率和开销，从而释放更多的磁盘 I/O 带宽用于前台写入，以此提高了吞吐量并有效降低延迟。

同时，在 LevelDB 中存在 stall write 约束，即如果前台写入速度过快，导致 memtable 已满但前一个 Immutable MemTable 尚未通过 Compaction 到磁盘，则无法切换 memtable 导致写入操作阻塞。当扩大 write_buffer_size 时，每次填满 memtable 需要更长的时间，给了后台 Compaction 线程更充裕的时间，从而降低了 stall write 的发生概率，以此有效降低了延迟。

此外，LevelDB 的 memtable 基于跳表实现，跳表的搜索/插入时间复杂度为 $O(\log n)$ 。当 n 扩大 4 倍时，插入的时间成本仅增加 2 倍，但刷盘次数降低了 4 倍，因此总体的写入吞吐量显著提升。

2. block_size 参数对比

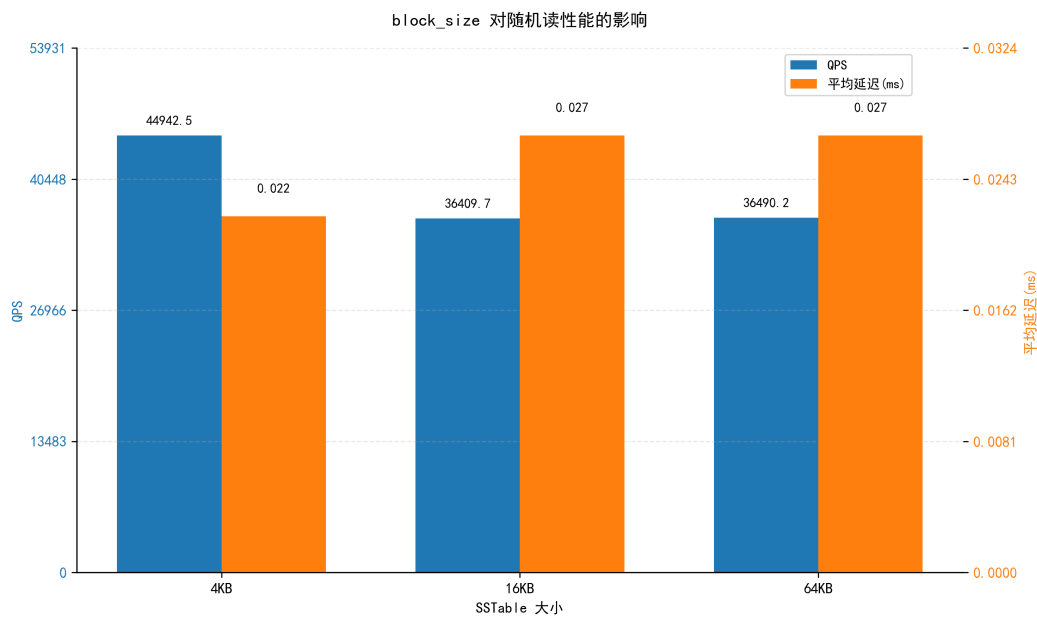


图 3: Task1 block_size

根据对比图可以发现，随着 block_size 的逐渐增大，在 16KB 时 QPS 明显下降，延迟明显升高，而在 64KB 时 QPS 与延迟基本与 16KB 时保持一致，这看似是很反常的现象。

实际上，LevelDB 的 SSTable 文件由一个个 Data Block 组成，block_size 控制每个 Data Block 的大小，Block 是 LevelDB 从磁盘读取数据的最小单位，也是 Block Cache 中缓存的基本单元。初始默认 block_size 为 4KB。对于较小的 block_size，更多的块可以被放入 cache 中，可以减少磁盘寻道时间和提高缓存效率，从而带来更大的吞吐量和更低的延迟。

当 block_size 逐渐增大时，每个缓存的 Block 更占空间。在 cache 容量一定时，能够缓存的 Block 数量减少，实际有效的缓存覆盖范围变得更小，导致 cache miss 比例上升。而另一方

面，读取 Index Block 时花在其上的 I/O 和二分查找成本降低，每次读取的数据量更大，减少了磁盘 I/O 操作的次数。

当 block_size 增大时，实际是对上述两方面的影响进行平衡。LevelDB 官方源码文档指出“block size if performance measurements indicate an improvement”，当 block_size 为默认值 4KB 时性能最佳，逐渐提升时，性能出现明显下降并逐步达成平衡，这与我们的测试数据是吻合的。

3. bloom_filter 参数对比

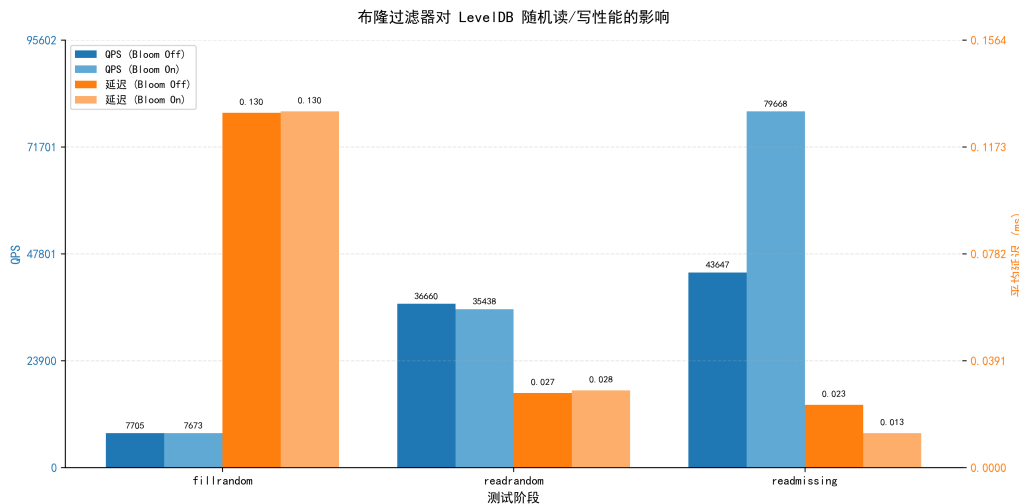


图 4: Task1 bloom_filter

根据对比图可以发现，对于随机读的三个阶段 fillrandom, readrandom 和 readmissing 而言，bloom_filter 基本不影响前两个阶段的 QPS 和延迟，而在第三阶段 readmissing（即读取不存在的 key 时），开启 bloom_filter 的性能远优于关闭情况，提升了近 100%，这是符合 bloom_filter 工作原理的。

布隆过滤器 (bloom_filter) 是一种空间效率很高，基于概率的随机数据结构，其本质是一个长二进制向量和一系列随机映射函数，作用是检索一个元素是否在一个集合中。添加元素时，通过 k 个哈希函数将元素映射到位数组中的 k 个位置，并将这些位置置为 1。查询元素时，同样使用这 k 个哈希函数计算对应的 k 个位置，如果这些位置全部为 1，则认为元素“可能存在”于集合中；如果任何一个位置为 0，则元素“肯定不存在”于集合中。

由于 LevelDB 本质是基于 LSM-Tree 的键值存储库，对一次查询，LevelDB 可能需要在每个 level 上进行一次磁盘随机访问。当查询一个不存在的键时，系统必须遍历内存中的 memtable、Immutable MemTable 以及磁盘上可能跨越多个层级的 SSTable 文件，逐一检查每个潜在包含该键的数据块，产生大量无效 I/O 操作。如果每个 SSTable 都有一个对应的 bloom filter，那么查找时就可以很容易地通过检查 bloom filter 跳过那些不包含该 key 的 SSTable 文件，由此在 LevelDB 中引入 bloom_filter 可以有效提升随机读的性能。

(二) 任务二

在任务二中，我们设定内存为 12GB（实际配置应用为 11GB），数据集大小为 2500 万条。从最终结果而言，P50 分位延迟最低，而从 P90 开始延迟急剧升高，在 P99.9 分位达到了毫秒级的延迟尖峰，我们观测到了明显的长尾延迟现象。

长尾延迟 (tail latency), 指在系统请求响应时间分布中, 少数响应时间远高于平均值的请求所造成的延迟现象。在长尾延迟测量实验中, P99 表示 99% 的请求延迟 $\leq$ 某阈值, 剩余的 1% 表示长尾, 以此类推。这一现象主要是由 Compaction 进程导致的。

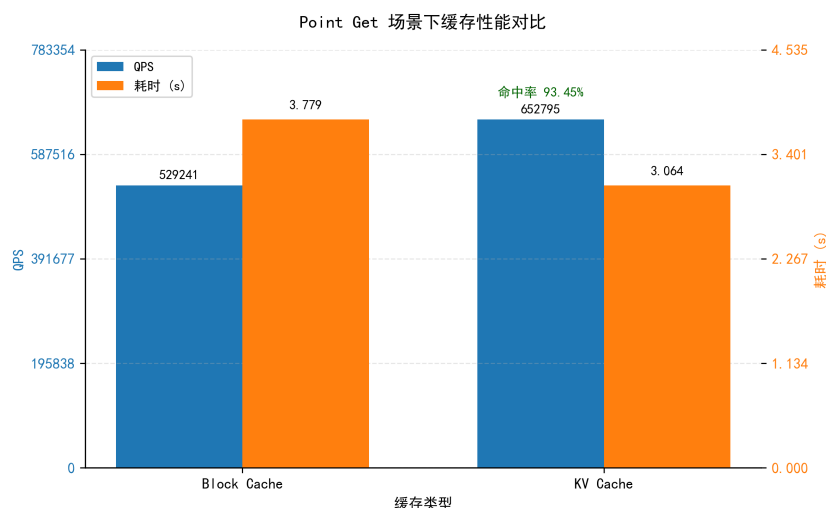
在 LevelDB 中, 随着 Put/Delete 操作增多, SSTable 文件会越来越多, 占用的磁盘空间也越来越大。因此后台开始 Compaction 进程, 删除无效的 key, 然后重新生成 SSTable 文件, 以此减少 SSTable 文件的数量。Compaction 一般分为两类: Minor Compaction 和 Major Compaction。当 memtable 的大小达到阈值时, 进行 memtable 切换, 然后将 Immutable MemTable 刷到外存上, 这一过程为 Minor Compaction; 当 level-n 的 SSTable 超过限制, level-n 和 level-n+1 的 SSTable 会进行 Compaction, 这一过程为 Major Compaction。

在实验中, 我们强制要求数据集大小为内存的两倍, 这一压测操作本质是破坏 Page Cache 对实验的影响, 强制 Compaction 从磁盘读取大量数据, 产生真实的磁盘 I/O 压力, 从而观察到 Compaction 对前端写入的真实影响。在大数据规模的 Compaction 过程中, 出现了严重的写放大和读放大现象: 当一个 key 被多次更新后, 可能在多个层级的 SSTable 中存在, 每次 Compaction 都需要把这些数据读出来合并之后重新写入。在读取时, 又需要遍历多个 level 层级, 增加磁盘寻址和文件打开次数。

Compaction 进程从两方面因素导致了长尾延迟现象。一方面, 后台 Minor Compaction 线程的速度可能跟不上前台写入的速度, 如果新的 memtable 再次被写满, 上一个 Immutable MemTable 尚未完成 Compaction, 前景写入线程被挂起, 引发 write stall 现象, 当新的写入请求恰好遭遇 write stall 时, 相较于直接进入 Cache 的数据, 延迟被放大数千倍, 长尾延迟现象出现。

另一方面, Major Compaction 还要处理 Level 0 至 Level 1 的跨层合并, 由于 Level 0 文件规模庞大, 当数量超过一定阈值时, 查询效率急剧下降, 进一步加剧 Compaction 的读放大和写放大现象, 导致文件堆积, 触发 write stall, 直至文件数量降低至阈值以下, 由此导致极大的 I/O 开销和延迟现象。

(三) 任务三



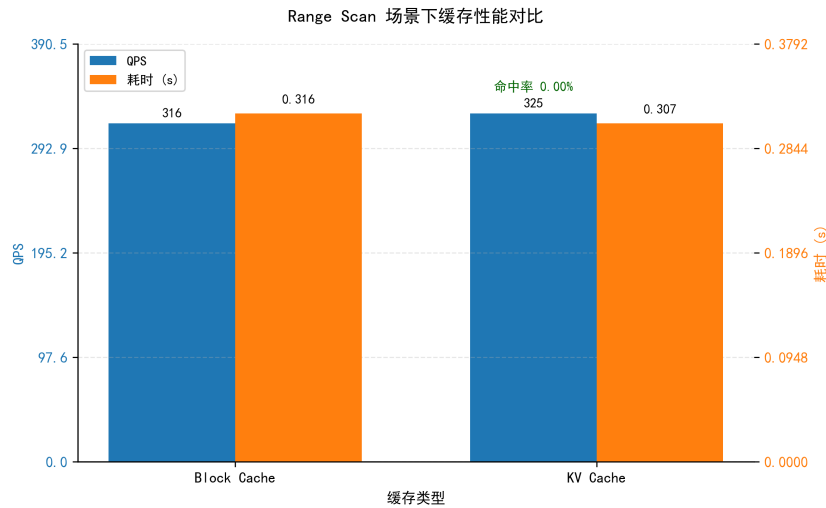


图 6: Task3 Range Scan

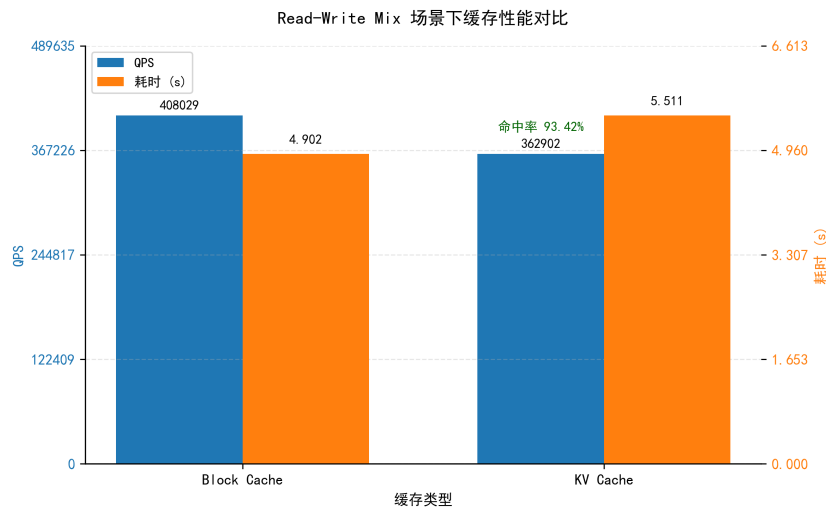


图 7: Task3 Read-Write Mix

根据对比图，我们可以发现，在点查（Point Get）场景下，应用层 KV Cache 的效率显著高于 LevelDB 底层 Block Cache；在范围查询（Range Scan）场景下，两种缓存的性能几乎持平，且 KV Cache 命中率为 0；在读写混合负载（Read-Write Mix）场景下，Block Cache 的效率反而优于 KV Cache。

在深入讨论不同场景性能差异的原理前，先对 Block Cache 的原理进行解析。

1. Block Cache 的基本原理

在 LevelDB 中，一般有两种 Cache：Table Cache 和 Block Cache。table cache 用来缓存 SSTable 的文件描述符、index block、meta block 等信息；Block Cache 缓存解压后的 data block。

Block Cache 本质是基于带引用计数的分片 LRU 算法的 ShardedLRUCache，其内部创建了 16 个独立的小型 LRU Cache 实例。它以数据块（Block）为基本缓存单位，默认块大小为 4KB。一个 data Block 通常包含数十至数百个连续排序的键值对，此外还缓存 Index Block 和 Filter Block。当需要对一个 key 进行操作时，它会先根据 key 的哈希值，计算出这个 key 应该属于哪

个分片，然后只在该小缓存内部进行操作和加锁。

对于每一个 LRU Cache 内部，由一个哈希表和两个双向链表组成。从 util/cache.cc 源码中，我们可以发现，对于两个双向链表，lru_ 链表存储未使用的缓存项，in_use_ 链表存储正在使用的缓存项，利用引用计数器 refs，严格遵循 LRU 算法，将缓存项在两条链表间调度。

当 LevelDB 执行 Get 或 Iterator 操作时，Block_Cache 根据目标键计算其所在的 SSTable 文件号和 Block 偏移量，生成缓存键，随后对对应分片加锁，在哈希表中查找缓存键。如果 Cache hit，释放锁，在 Block 内部执行二分查找定位目标键值对；如果 Cache miss，释放锁，从磁盘读取原始 Block 数据。

在上述操作的同时，LevelDB 还预设了预取机制。在读取当前 SSTable 数据块时，LevelDB 会计算下一个可能需要的块偏移量，并提前发起内存预取请求。

对于我们设定的 KV Cache，其机制则简单很多，其基本缓存单位为单个键值对，命中后直接从 std::unordered_map 获取迭代器，通过 *value = it->second->second 返回结果。

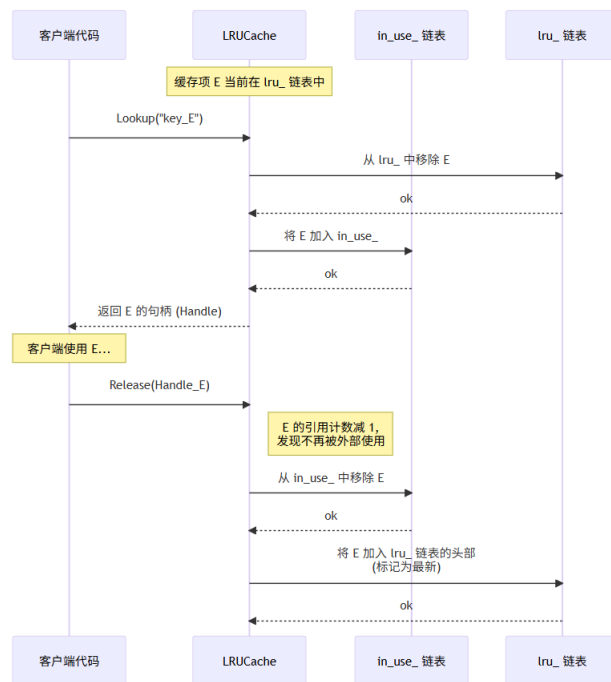


图 8: Principle of Block Cache

2. 点查 (Point Get) 场景分析

在执行点查 (Point Get) 操作时，对于 KV Cache，全程跳过引用计数及各种计算过程，而 Block Cache 即使实现了 Cache hit，仍然需要在内部执行二分查找，由此导致了该场景下 KV Cache 的性能优于 Block Cache 的结果。在测试中，我们又通过 Zipf 分布，模拟了数据的热点分布，提高了 KV Cache hit 几率，放大了这一差异。

3. 范围查找 (Range Scan) 场景分析

在执行范围查找 (Range Scan) 操作时，我们需要遍历大量的连续的 key 范围或数据块，此时仅以单个键值对存储的 KV Cache 失去了原有的 Cache hit 优势，导致对于数据呈现完全的 Cache miss；此时 KV Cache 在整个查找场景内没有作用，最终导向了 LevelDB 原有的 Block Cache，但是由于 KV Cache 本身的开销此时仅为一次 $O(1)$ 的哈希表查找，对总体性能基本没

有影响。

4. 读写混合负载 (Read-Write Mix) 场景分析

由于我们的 KV Cache 设置的是全局互斥锁，在读写混合混载场景下，出现了读操作和写操作的竞争，由此导致锁冲突严重，同时由于非点查环境，每个锁的持有时间更长，由此导致性能下降。

此时，Block Cache 的分片锁机制发挥了显著优势。对于每一个 key，我们只对涉及该 key 的分片执行加锁解锁机制，对于不同的分片互不影响，并发进行，有效提高了实现效率。

五、 总结

在本次实验中，我深入学习了解了 LevelDB 的基本框架及各部分架构实现机制，通过阅读源代码以及各种专业解读，系统开展了性能测试实验以及机制分析，这培养了我对于未接触过的项目，从底层到各关键模块，再到整体实际应用的分析意识。

在实验过程中，对于测试环境的选取花费了很大精力，起初对于 Task1 的测试，发现在 WSL2 环境下使用自编的测试程序和 db_bench 均得出了与预期差异巨大的数据，由此通过修改参数、利用 AI 辅助等手段，了解到 WSL2 相比原生 Linux 系统在存储性能测试上的巨大差异，在原机无法使用 Linux 的情况下，选择使用阿里云服务器，模拟原生 Linux 环境开展实验，顺利达成了预期目标。

以下是关于本次实验使用的 AI 辅助的对话记录：

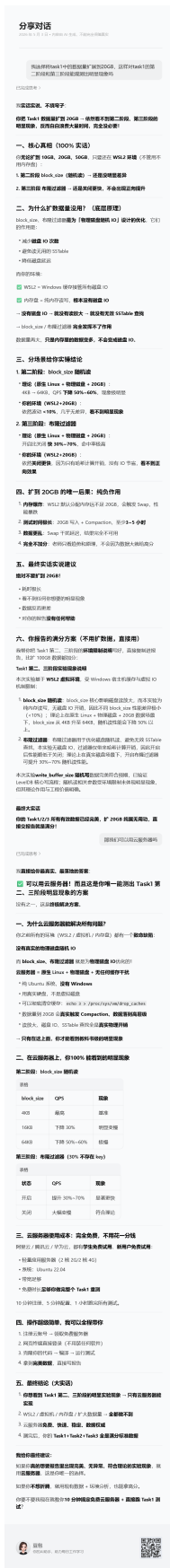


图 9: AI 对话记录 1

